

EduTrack AI

Architecture & Workflow Documentation

Hackathon MVP · Personalized, AI-powered college learning

Purpose

A concise technical overview for the GitHub repository, explaining how the current React MVP is structured, how users move through the application, and how learning features interact.

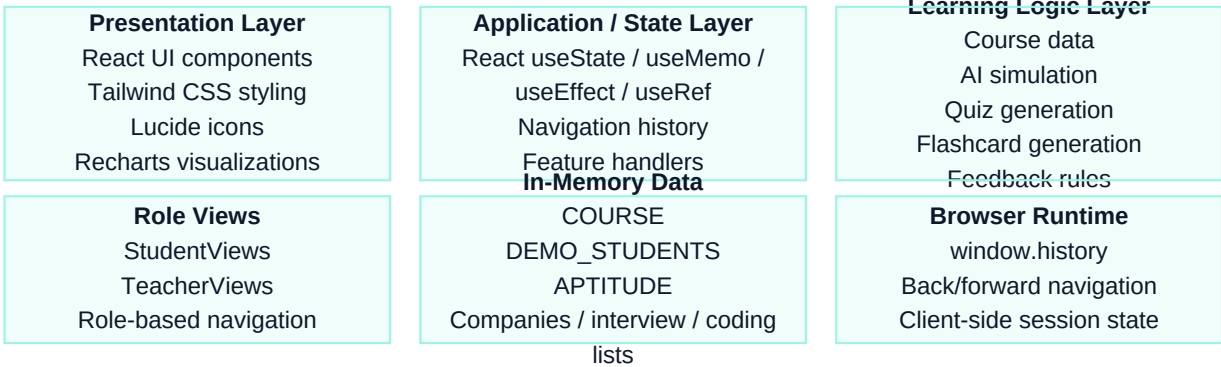
Source basis

This document is derived from the supplied **EduTrackAI (1).jsx** implementation. The current implementation is a client-side MVP with demo data and simulated AI behavior; this document does not assume a backend or external AI API that is not present in the source.

Key implementation evidence: React state and view routing are centralized in the root App, while StudentViews and TeacherViews dispatch to feature-specific components. ■filecite■turn1file0■L28-L47■ ■filecite■turn1file6■L320-L329■

1. System Architecture

EduTrack AI follows a single-page React application architecture. The root **App** component manages authentication state, navigation state, learner progress, notes, reminders, quiz results, language selection, and teacher/student mode. The UI is then delegated to role-specific view routers.



The source imports React hooks, Lucide React icons, and Recharts chart components. `filecite turn0file0 L1-L12`

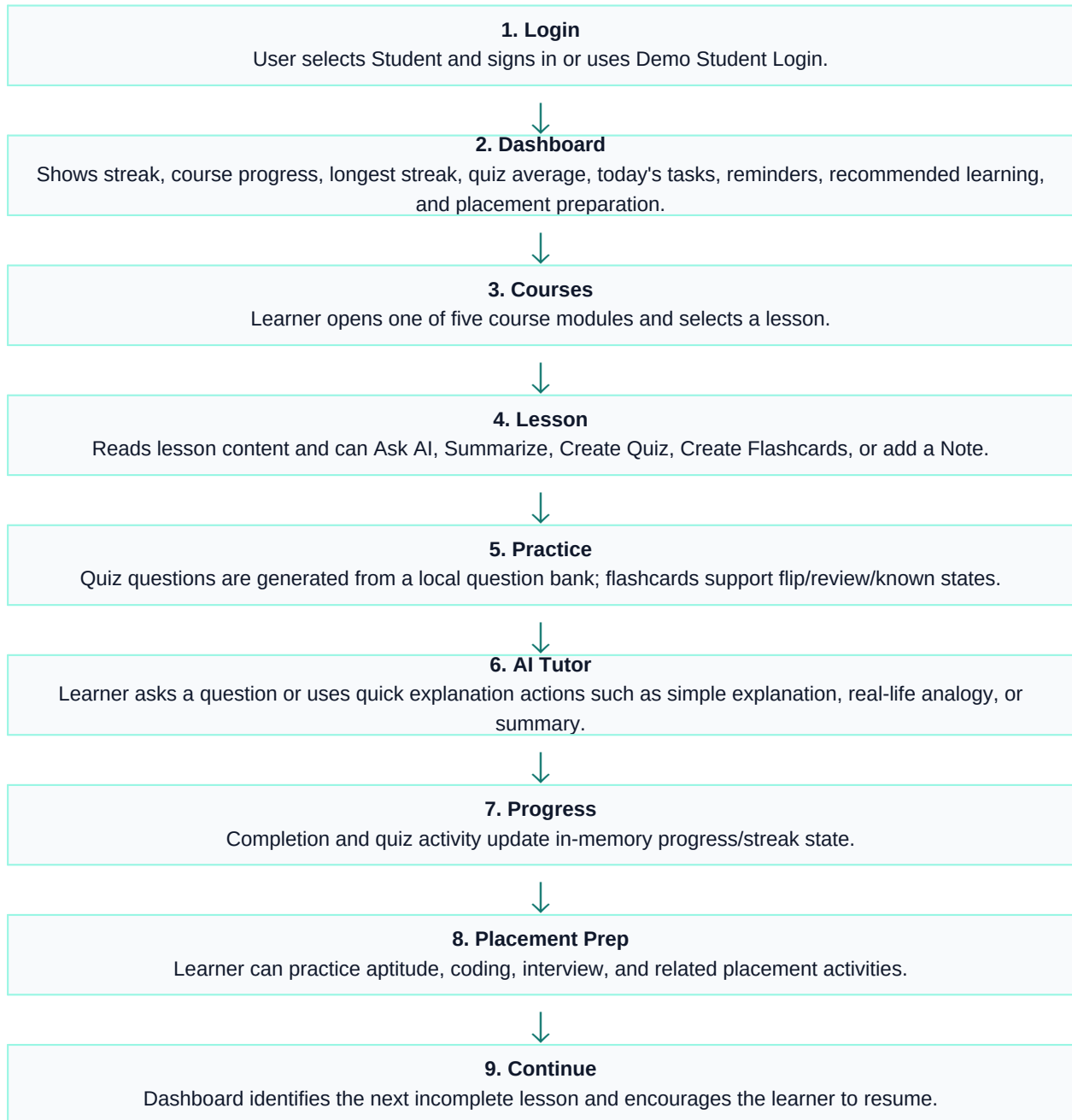
2. High-Level Component Structure

Layer / Component	Responsibility
App	Root shell; authentication gate; global state; navigation history; responsive layout.
LoginScreen	Select Student or Teacher role, collect login fields, and provide demo login.
StudentViews	Routes student view keys to Dashboard, Courses, Lesson, AI Tutor, Quizzes, Flashcards, Notes, General Learning, Placement, Progress, Reminders, and Profile.
TeacherViews	Routes teacher view keys to Dashboard, Students, Reports, AI Insights, and Profile.
Feature components	Implement each learning, analytics, placement, notes, reminder, and profile workflow.

The root App explicitly passes learning state and handlers into StudentViews, and role selection determines whether StudentViews or TeacherViews is rendered. `filecite turn1file3 L167-L182`

3. Student Workflow

The primary learner journey is designed around a loop of **learn** → **practice** → **review** → **track progress** → **continue learning**.



The dashboard calculates the next incomplete lesson from COURSE modules and the completedLessons set, and uses reminders to surface upcoming tasks. ■filecite■turn1file7■L399-L413■

4. AI & Learning Workflow

In the current MVP, the AI layer is **simulated locally**. It does not call an external LLM service in the supplied implementation.

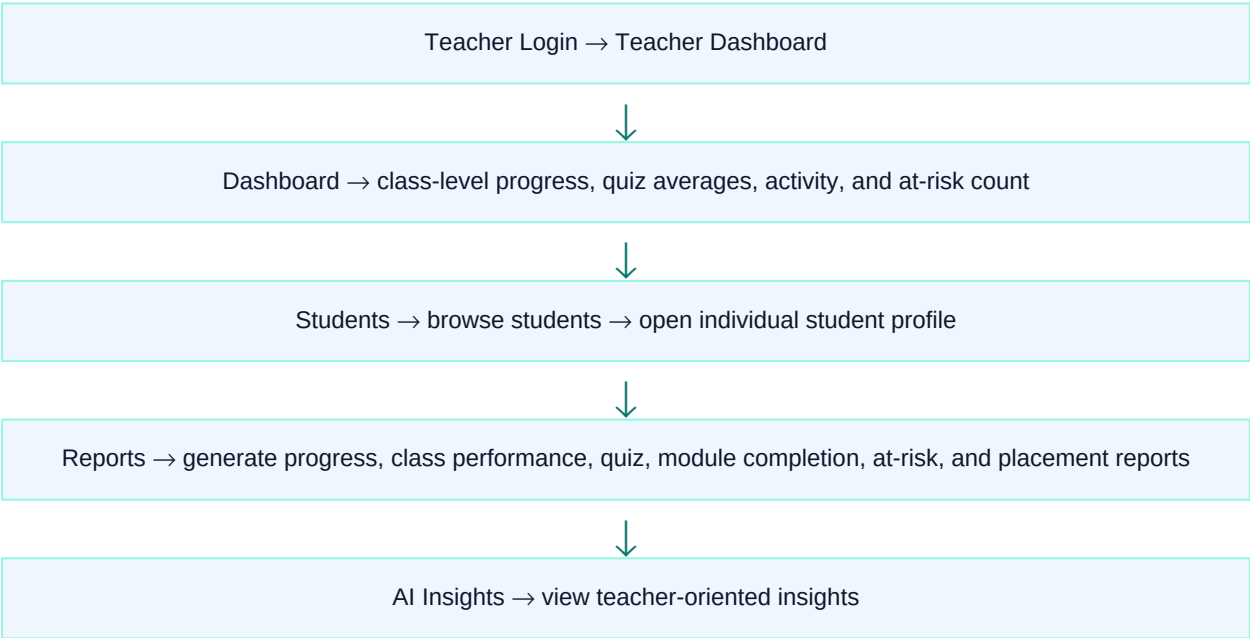
Input	Local processing	Output
Lesson/topic + language	aiExplain(topicTitle, mode, language)	Simple explanation / real-life analogy / summary
Student question	simulateChatResponse(question, topic, language)	Contextual demo tutor response
Topic + quiz count + difficulty	generateQuiz(topicTitle, count, difficulty)	MCQ / short-answer quiz items
Topic	generateFlashcards(topicTitle)	Five flashcards with front/back content
Answer text	feedbackFor(answerText)	Clarity, structure, grammar, and improvement suggestion

The implementation defines explanation templates for English, Tamil, and Hindi, with a fallback explanation generator for other topics. [filecite\[turn0file0\]L120-L167](#)

The AI Tutor appends the learner message and then returns a simulated response after a short timeout, while quick actions call the local explanation functions. [filecite\[turn1file8\]L453-L475](#)

5. Teacher Workflow

Teacher mode focuses on monitoring and intervention rather than consuming lessons.



Teacher reports are calculated from DEMO_STUDENTS and COURSE data. For example, the report layer supports student progress, class performance, quiz performance, module completion, at-risk students, and placement preparation. [filecite\[turn1file4\]L233-L246](#)

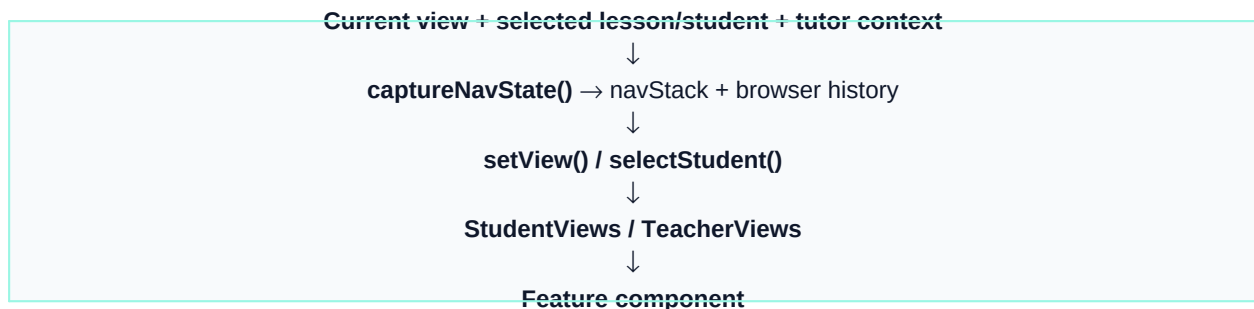
6. Data Model in the Current MVP

Data object	Purpose / examples
COURSE	Course → modules → lessons; current sample course is Computer Science Fundamentals with five modules.
DEMO_STUDENTS	Demo learner profiles containing progress, streaks, quiz averages, weak topics, placement progress, and time spent.
APTITUDE	Quantitative Aptitude, Logical Reasoning, and Verbal Ability topic lists.
COMPANIES	Placement company examples: Infosys, Cognizant, Google, Microsoft, Amazon.
CODING_LANGS	C, C++, Java, Python, SQL, Data Structures, Algorithms.
INTERVIEW_QUESTIONS	Common interview practice questions.
React state	completedLessons, streak, notes, reminders, quizResults, selected module/lesson, tutor context, selected student, etc.

The supplied source defines course modules and lessons, aptitude categories, placement companies, coding languages, interview questions, and demo student records directly in the JavaScript module. ■filecite■turn0file0■L16-L19■ ■filecite■turn0file0■L91-L110■

7. Navigation & State Flow

Navigation is intentionally implemented inside the client application. A custom `setView` function records the current screen in a stack and also pushes a browser history entry. Browser back/popstate restores the previous view snapshot. An unfinished quiz can trigger a confirmation before navigation continues.

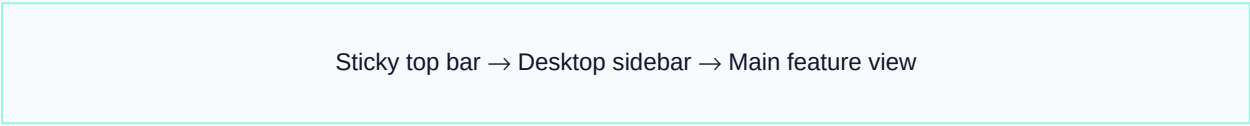


The source explicitly stores navigation snapshots and handles window popstate, including a guard for unfinished quizzes. ■filecite■turn0file0■L336-L394■

8. UI & Responsive Architecture

The application uses a shared visual shell with a sticky top bar, desktop sidebar, mobile navigation drawer, mobile bottom navigation, cards, modals, progress bars, toast notifications, dark mode, and responsive grid layouts.

Desktop



Mobile



The root layout contains the top bar, desktop sidebar, mobile drawer, main content area, and mobile bottom navigation.

filecite turn0file0 L442-L535

9. Current MVP Boundaries

- Authentication is a UI/demo flow; the supplied code does not implement server-side credential verification.
- Course, student, aptitude, company, coding, and interview data are embedded demo data.
- AI explanations, tutor responses, quizzes, flashcards, and feedback are generated locally by JavaScript functions.
- Application state is maintained with React state during the session; the supplied source does not show a persistent database layer.
- The architecture is therefore best described as a front-end MVP/prototype rather than a production multi-tier system.

10. Suggested Production Evolution

For a production release, the same UI architecture can be retained while introducing service boundaries:

Current MVP	Production extension
Local demo login	Auth service + secure session/token management
In-memory course/student data	REST/GraphQL API + relational database
Local AI simulation	LLM service/API + prompt/context layer + safety/usage controls
Local quiz generation	Question bank service + adaptive assessment engine
Local progress state	Persistent learner progress/event store
Teacher reports from demo arrays	Analytics service + scheduled reports + role-based access
Client-only reminders	Notification/reminder service with email/push integration

GitHub README summary

EduTrack AI is a React-based educational platform MVP that combines structured course learning, AI-assisted explanations, quizzes, flashcards, notes, progress tracking, reminders, and placement preparation for students, alongside analytics, reports, and AI insights for teachers.

This architecture document is intended to accompany the source code in a GitHub repository and can be used as a project documentation PDF.